

SCHOOL OF INFORMATION SCIENCE PRESIDENCY UNIVERSITY

Rajanakunte, Bangalore



PRESIDENCY UNIVERSITY

Course Title: Big Data Analytics Lab

L- T-P- C: 2-0-2-3

Course Code: CSA 3004

General Lab Etiquette

Do's for Students in the Lab

1. **Be Punctual:** Arrive on time and be prepared with the necessary materials.
2. **Follow Lab Guidelines:** Familiarize yourself with and follow all lab rules and guidelines, including safety protocols and usage policies.
3. **Ask for Help if Needed:** Don't hesitate to ask the instructor or lab technician for assistance if you encounter problems with equipment or software.
4. **Keep the Workspace Clean:** Tidy up the area before leaving; remove any trash, and place chairs and equipment back in their designated positions.
5. **Handle Equipment Carefully:** Use all lab equipment with care and follow instructions to avoid damage or disruption.
6. **Report Issues:** Immediately report any malfunctioning equipment or software issues to the lab supervisor or technician.
7. **Respect Other Users:** Maintain a quiet environment, minimize distractions, and respect the privacy and workspace of others.
8. **Practice Time Management:** Use your time wisely and complete assigned tasks within the lab session.

Don'ts for Students in the Lab

1. **Don't Bring Food or Drinks:** Avoid bringing food or drinks into the lab to prevent spills that could damage equipment.
2. **Don't Download Unauthorized Software:** Do not download or install unauthorized software, games, or other applications that could compromise lab computers.
3. **Don't Access Unauthorized Files or Data:** Respect privacy and avoid accessing files or folders that do not belong to you.
4. **Don't Use Lab Equipment for Personal Use:** Avoid using lab computers and equipment for non-academic activities (e.g., social media, online shopping) unless specifically permitted.
5. **Don't Disrupt Others:** Avoid loud conversations, music, or any behaviour that may distract other students working in the lab. Don't use electronic gadgets (e.g., Mobile phone, Tab, etc.) during the lab hours.

Course Outcome	Topic
CO1	Describe the fundamental concepts of big data analytics [Understand]
CO2	Apply Map-Reduce programming on the given datasets to extract required insights. [Apply]
CO3	Employ appropriate Hadoop Ecosystem tools such as Hive, Hbase to perform data analytics for a given problem [Apply]
CO4	Use Spark tool to analyse the given dataset for a given problem. [Apply]

Programs to be covered according to Course Handout

Week 1	1. Installation of Hadoop single node cluster using Ubuntu operating system
	2. Imagine you're managing a large-scale data processing task for a company that deals with huge volumes of data. You're responsible for using Hadoop to handle and analyze this data. In this context, you frequently need to execute various Hadoop commands to perform tasks like storing data, querying datasets, and managing distributed storage. Your goal is to efficiently leverage these commands to ensure smooth and effective data processing operations.
Week 2	3. Imagine you're working on a project where you need to analyze a large collection of text documents to determine the frequency of each word. To handle this efficiently, you decide to use a MapReduce approach. In this scenario, you'll use a sample dataset of text documents. Your goal is to implement a MapReduce job that processes these documents to count how many times each word appears across the entire dataset.
	4. Imagine you're working for a financial analytics company that wants to optimize its stock analysis process. The company has collected a vast amount of stock market data over the past decade. To efficiently analyze this data and extract valuable insights, you decide to use a MapReduce approach. You start by splitting the data into manageable chunks, then apply the Map function to process each

	chunk, and finally use the Reduce function to aggregate and summarize the results. This way, you can handle large datasets and perform complex stock analysis quickly and effectively.
Week 3	5. "Imagine you're working for a company that needs to understand the web traffic patterns on their site. To do this, you decide to use a sample dataset of web logs. By employing MapReduce, you can efficiently process and analyze this data to uncover insights about user behavior and site performance. How would you approach this scenario?" 6. Imagine you're working for a weather forecasting company that needs to analyze temperature data collected from various weather stations across the country. You have a large dataset with temperature readings from thousands of stations. To make sense of this massive amount of data, you decide to use a MapReduce approach.
Week 4	Revision
Week 5	Lab Test – 1
Week 6	7. You're managing a large dataset of customer transactions and need to analyze it using Hive 8. Imagine you're working as a data analyst at a company that deals with large datasets stored in Hadoop. You've been tasked with optimizing queries and extracting valuable insights from these datasets. To achieve this, you need to use advanced Hive commands to efficiently manage and analyze the data.
Week 7	9. Imagine you're working for a company that maintains a large customer database. You've been tasked with using HBase to perform various operations on this database. Here's how you might approach it with some basic HBase commands. 10. Imagine you're working on a large-scale data processing project where you need to efficiently manage and interact with a massive HBase database. 11. Install, Deploy & configure Apache Spark

Week 8	12. You're working on a text analysis project for a large publishing company. They've asked you to develop a solution using Apache Spark to help them analyze a collection of text files.
	13. Write a MapReduce program to create an index from a collection of documents to allow for fast keyword-based search.
Week 9	14. Write a MapReduce program to analyze relationships and interactions in a social network graph (e.g., identifying communities or influential nodes).
	Revision
Week 10	Lab Test – 2

1. Installation of Hadoop single node cluster using Ubuntu operating system.

STEP-1 SOFTWARES REQUIRED

Hadoop 1.2.0-bin.tar.gz

Jdk 7u67-linux-i586.tar.gz

Link for JDK

<http://www.oracle.com/technetwork/java/javase/downloads/index.html>

Link of Hadoop : <http://mirror.fibergrid.in/apache/hadoop/common/hadoop-1.2.0>

STEP-2 Untar the software

Open the terminal window

Type the command : cd Desktop

Type ls command

```
tar -zxvf jdk-7u67-linux-i586.tar.gz
```

```
tar -zxvf hadoop-1.2.0-bin.tar.gz
```

STEP -3 Bashrc configurations

Open the terminal type the command

```
sudo gedit ~/.bashrc
```

```
export JAVA_HOME=/home/user/Desktop/jdk1.7.0_67
```

```
export PATH=$PATH:$JAVA_HOME/bin
```

```
export HADOOP_HOME=/home/user/Desktop/hadoop-1.2.0
```

```
export PATH=$PATH:$HADOOP_HOME/bin
```

Open new terminal and check for java version and Hadoop version

STEP- 4 Hadoop Configuration Files

1. Core-site.xml

(Configuration setting for hadoop core, such as I/O setting that are common to HDFS and Mapreduce)

2.Hdfs-Site.xml

(Configuration setting for HDFS daemons the name node, Secondary name

node and data node and Replication factor as well)

3. Mapred-site.xml

(configuration setting for MapReduce daemons: the job tracker and the task tracker)

4. Hadoop-Env_sh

(Environment variables that are used in the scripts to run Hadoop)

- Open the new terminal window
- Go to the hadoop folder
- cd Desktop
- cd Hadoop (Press tab) for path
- cd conf
- Type ls
- All the above mentioned files are listed

- Go to terminal and type

sudo gedit core-site.xml

```
<configuration>
```

```
<property>
```

```
<name>fs.default.name</name>
```

```
<value>hdfs://localhost:9000</value>
```

```
</property>
```

```
</configuration>
```

Ip address :127.0.0.0 , 198.1.27.0

Port no: 80,

- Go to terminal and type

sudo gedit hdfs-site.xml

```
<configuration>
```

```
<property>
```

```
<name>dfs.replication</name>
```

```
<value>1</value>
```

```
<name>dfs.name.dir</name>
```

```
<value>/home/user/Desktop/hadoop-1.2.0/name/data</value>
```

```
</property>
```

```
</configuration>
```

- Go to terminal and type
sudo gedit mapred-site.xml
<configuration>
<property>
<name>mapred.job.tracker</name>
<value>hdfs://localhost:9001</value>
</property>
</configuration>
- Go to terminal and type
sudo gedit hadoop-env.sh
export JAVA_HOME=/home/user/Desktop/jdk1.7.0_67

STEP 5 SSh Configuration

- sudo apt-get install ssh
- ssh-keygen -t rsa -P ""
Sharing the public key with host
- ssh-copy-id -i ~/.ssh/id_rsa.pub user@ubuntuvn and check
- ssh localhost (It should not ask password)

STEP – 6 Formatting the Name Node

- \$ hadoop namenode -format
- format a new distributed file system
- Process creates an empty file system for creating the storage directories

STEP-7&8 Starting Hadoop Daemons

Open the terminal window and type
\$ start-all.sh
and type


```
$ jps
Hadoop dfs ls/ //deprecated
Hadoop dfs -mkdir krishna/ //directory created
Hadoop dfs -mkdir/Krishna/abc/
Now check if directory is there
Hadoop dfs -ls/Krishna
Found 1 item
```

- Copy any file n paste in desktop
- Hadoop dfs put
- **user@ubuntuvn:~\$ start-all.sh**
- **Warning: \$HADOOP_HOME is deprecated.**
- **starting namenode, logging to /home/user/Downloads/hadoop-1.2.0/libexec/./logs/hadoop-user-namenode-ubuntuvn.out**
- **localhost: datanode running as process 2978.**
- **localhost: secondarynamenode running as process 3123.**
- **jobtracker running as process 3204.**
- **localhost: tasktracker running as process 3342.**
- **user@ubuntuvn:~\$ jps**
- **4020 Jps**
- **3342 TaskTracker**
- **3204 JobTracker**
- **3123 SecondaryNameNode**
- **3606 NameNode**
- **2978 DataNode**

2. Hadoop Commands

Open Terminal in Cloudera and start typing the below Commands:

2. **\$ hostname**
3. **\$ sudo /home/cloudera/cloudera-manager - -express - -force**
4. **\$ service hadoop-hdfs-namenode restart**

←- The above 3 commands were to start hadoop→

5. **\$ hadoop fs -ls /**
6. **\$ hdfs dfs -ls /**

←- The above 4 & 5 commands were to list hadoop files→

←- The below commands were to execute hadoop commands→

7. **\$ sudo -u hdfs hadoop dfs -mkdir /Ayesha**
8. **\$ hdfs dfs -ls /**
9. **\$ hdfs dfs -lsr /**
10. **\$ sudo -u hdfs hadoop dfs -touchz / Ayesha /myfile.txt**
11. **\$ gedit kk.txt** ----- enter your contents, save and close the file ---
--
12. **\$ sudo -u hdfs hadoop dfs -put kk.txt / Ayesha**
13. **\$ sudo su** ----- enter root-----
14. **\$ hadoop fs -get /Ayesha /kk.txt /root/Desktop**
15. **\$ exit** -----exit from root -----
16. **\$ sudo -u hdfs hadoop dfs -mkdir /taru**
17. **\$ sudo -u hdfs hadoop dfs -cp / Ayesha /taru**
18. **\$ sudo -u hdfs hadoop dfs -touchz / Ayesha /PP.txt**
19. **\$ sudo -u hdfs hadoop dfs -mv / Ayesha /PP.txt /taru**
20. **\$ sudo -u hdfs hadoop dfs -rm / Ayesha**
21. **\$ sudo -u hdfs hadoop dfs -du /taru**
22. **\$ sudo -u hdfs hadoop dfs -dus /taru**
23. **\$ sudo -u hdfs hadoop dfs -setrep -R -w 4 /taru/PP.txt**
24. **\$ sudo -u hdfs hadoop dfs -stat /taru**
25. **\$ sudo -u hdfs hadoop dfs -rm /taru/PP.txt**
26. **\$ sudo -u hdfs hadoop dfs -rmr /taru**
27. **\$ service hadoop-hdfs-namenode stop**

3. Write a Program to Perform Word Count using Map Reduce

Word Count Program

1. Add Hadoop Java jar file

File – new – Java Project – Name:wc1 – finish

Wc1 – right Click – Build Path – configure BD – Libraries – Add external jars – Select Desktop – Hadoop_java_jars – ctrl A – ok – ok

2. Create 3 classes

- Right Click On Project(wc1) – new – class – Name:wordcountMain – method – public SVM – finish
- Right Click On Project(wc1) – new – class – Name:wcreducer – method – inherited....
- Right Click On Project(wc1) – new – class – Name:wcmapper – method – inherited....

3. Create Jar Files

Right Click On Project(wc1) – Export – jar Files – Path – Browse – Select directory (desktop) – press ok

Give File Name: WC.jar – ok – finish

4. To upload input file from local system to hdfs directory:

1. Open The Terminal and type
2. `hadoop dfs -mkdir /WCA`

Name node is in safe mode.

3. `hadoop dfsadmin -safemode leave`
4. `hadoop dfs -put /home/training/Desktop/words1.txt /WCA/w1`
5. `hadoop jar /home/training/Desktop/wcA.jar wordcountMain /WCA/w1 /wordcountout`

To Check The Output –

Open Firefox and Browse and type

Hadoop Administration

Browse the file System

Word count main

```
import org.apache.hadoop.conf.Configured;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.FileInputFormat;
import org.apache.hadoop.mapred.FileOutputFormat;
import org.apache.hadoop.mapred.JobClient;
import org.apache.hadoop.mapred.JobConf;
import org.apache.hadoop.util.Tool;
import org.apache.hadoop.util.ToolRunner;

public class wordcount extends Configured implements Tool {

    @Override
    public int run(String[] args) throws Exception {

        if(args.length<2)
        {
            System.out.println("Plz Give Input Output Directory Correctly");
            return -1;
        }

        JobConf conf = new JobConf(wordcount.class);

        FileInputFormat.setInputPaths(conf,new Path(args[0]));
        FileOutputFormat.setOutputPath(conf, new Path(args[1]));

        conf.setMapperClass(wordmapper.class);
        conf.setReducerClass(wordreducer.class);

        conf.setMapOutputKeyClass(Text.class);
        conf.setMapOutputValueClass(IntWritable.class);
        conf.setOutputKeyClass(Text.class);
        conf.setOutputValueClass(IntWritable.class);
```

```
        JobClient.runJob(conf);
        return 0;
    }

    public static void main(String args[]) throws Exception
    {
        int exitcode = ToolRunner.run(new wordcount(), args);
        System.exit(exitcode);
    }
}
```

Wordmapper

```
import java.io.IOException;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.MapReduceBase;
import org.apache.hadoop.mapred.Mapper;
import org.apache.hadoop.mapred.OutputCollector;
import org.apache.hadoop.mapred.Reporter;

public class wordmapper extends MapReduceBase implements
Mapper<LongWritable,Text,Text,IntWritable>
{
    public void map(LongWritable key, Text value,
        OutputCollector<Text, IntWritable> output, Reporter r)
        throws IOException
    {
        String s =value.toString();
        for(String word:s.split(" "))
        {
            if(word.length()>0)
            {
                output.collect(new Text(word), new IntWritable(1));
            }
        }
    }
}
```

```
        }  
  
    }  
  
}
```

Word reducer

```
import java.io.IOException;  
import java.util.Iterator;  
  
import org.apache.hadoop.io.IntWritable;  
import org.apache.hadoop.io.Text;  
import org.apache.hadoop.mapred.MapReduceBase;  
import org.apache.hadoop.mapred.OutputCollector;  
import org.apache.hadoop.mapred.Reducer;  
import org.apache.hadoop.mapred.Reporter;  
  
public class wordreducer extends MapReduceBase implements  
    Reducer<Text,IntWritable,Text,IntWritable>  
    {  
        public void reduce(Text key, Iterator<IntWritable> values,  
            OutputCollector<Text, IntWritable> output, Reporter r)  
            throws IOException {  
  
            int count=0;  
            while(values.hasNext())  
            {  
                IntWritable i= values.next();  
                count+= i.get();  
            }  
            output.collect(key, new IntWritable(count));  
        }  
    }
```

4. Write a Program to Perform Stock Analysis using Map Reduce

Main

```
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.conf.Configured;
import org.apache.hadoop.io.FloatWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.FileInputFormat;
import org.apache.hadoop.mapred.FileOutputFormat;
import org.apache.hadoop.mapred.JobClient;
import org.apache.hadoop.mapred.JobConf;
import org.apache.hadoop.util.Tool;
import org.apache.hadoop.util.ToolRunner;

public class MaxClosePrice extends Configured implements Tool {

    public int run(String[] args) throws Exception {
        if(args.length<2)
        {
            System.out.println("Plz Give Input Output Directory
Correctly");
            return -1;
        }
        JobConf conf = new JobConf(MaxClosePrice.class);
        FileInputFormat.setInputPaths(conf,new Path(args[0]));
        FileOutputFormat.setOutputPath(conf, new Path(args[1]));
        conf.setMapperClass(MaxClosePriceMapper.class);
        conf.setReducerClass(MaxClosePriceReducer.class);
        conf.setMapOutputKeyClass(Text.class);
        conf.setMapOutputValueClass(FloatWritable.class);
        conf.setOutputKeyClass(Text.class);
        conf.setOutputValueClass(FloatWritable.class);
        JobClient.runJob(conf);
        return 0;
    }
}
```

```
        public static void main(String[] args) throws Exception {
            int exitcode = ToolRunner.run(new MaxClosePrice(), args);
            System.exit(exitcode);
        }
    }
```

Mapper

```
import java.io.IOException;

import org.apache.hadoop.io.FloatWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.MapReduceBase;
import org.apache.hadoop.mapred.OutputCollector;
import org.apache.hadoop.mapred.Reporter;
import org.apache.hadoop.mapred.Mapper;

public class MaxClosePriceMapper extends MapReduceBase implements
Mapper<LongWritable, Text, Text, FloatWritable>
{

    @Override
    public void map(LongWritable key, Text value,
                    OutputCollector<Text, FloatWritable> output, Reporter r)
                    throws IOException {

        String line = value.toString();
        String[] items = line.split(",");

        String stock = items[1];
        Float closePrice = Float.parseFloat(items[6]);

        output.collect(new Text(stock), new FloatWritable(closePrice));

    }
}
```



```
}
```

Reducer

```
import java.io.IOException;
import java.util.Iterator;

import org.apache.hadoop.io.FloatWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.MapReduceBase;
import org.apache.hadoop.mapred.OutputCollector;
import org.apache.hadoop.mapred.Reducer;
import org.apache.hadoop.mapred.Reporter;

public class MaxClosePriceReducer extends MapReduceBase implements
Reducer<Text,FloatWritable,Text,FloatWritable>
{

    @Override
    public void reduce(Text key, Iterator<FloatWritable> values,
        OutputCollector<Text, FloatWritable> output, Reporter r)
        throws IOException {

        float maxClosePrice = 0;

        //Iterate all and calculate maximum
        while (values.hasNext()) {
            FloatWritable i = values.next();
            maxClosePrice = Math.max(maxClosePrice, i.get());
        }

        //Write output
        output.collect(key, new FloatWritable(maxClosePrice));
    }
}
```

5. Write a Program to Perform Log Analysis using Map Reduce

Main

```
import java.io.*;
import org.apache.hadoop.conf.Configured;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.FileInputFormat;
import org.apache.hadoop.mapred.FileOutputFormat;
import org.apache.hadoop.mapred.JobClient;
import org.apache.hadoop.mapred.JobConf;
import org.apache.hadoop.util.Tool;
import org.apache.hadoop.util.ToolRunner;

public class ProcessLogs extends Configured implements Tool {

    @Override
    public int run(String[] args) throws Exception {

        if(args.length<2)
        {
            System.out.println("Plz Give Input Output Directory Correctly");
            return -1;
        }

        JobConf conf = new JobConf(ProcessLogs.class);
        FileInputFormat.setInputPaths(conf,new Path(args[0]));
        FileOutputFormat.setOutputPath(conf, new Path(args[1]));
        conf.setMapperClass(LogMapper.class);
        conf.setReducerClass(LogReducer.class);
        conf.setMapOutputKeyClass(Text.class);
        conf.setMapOutputValueClass(IntWritable.class);
        conf.setOutputKeyClass(Text.class);
        conf.setOutputValueClass(IntWritable.class);
```

```
        JobClient.runJob(conf);
        return 0;
    }

    public static void main(String args[]) throws Exception
    {
        int exitcode = ToolRunner.run(new ProcessLogs(), args);
        System.exit(exitcode);
    }
}
```

Mapper

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.MapReduceBase;
import org.apache.hadoop.mapred.Mapper;
import org.apache.hadoop.mapred.OutputCollector;
import org.apache.hadoop.mapred.Reporter;

public class LogMapper extends MapReduceBase implements
Mapper<LongWritable,Text,Text,IntWritable>
{
    public void map(LongWritable key, Text value,
        OutputCollector<Text, IntWritable> output, Reporter r)
        throws IOException {

        String[] s = value.toString().split(" ");
        String ip = s[0];

        output.collect(new Text(ip), new IntWritable(1));
    }
}
```

Reducer

```
import java.io.IOException;
import java.util.Iterator;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.MapReduceBase;
import org.apache.hadoop.mapred.OutputCollector;
import org.apache.hadoop.mapred.Reducer;
import org.apache.hadoop.mapred.Reporter;

public class LogReducer extends MapReduceBase implements
Reducer<Text,IntWritable,Text,IntWritable>
{

    public void reduce(Text key, Iterator<IntWritable> values,
        OutputCollector<Text, IntWritable> output, Reporter r)
        throws IOException {

        int count=0;
        while(values.hasNext())
        {
            IntWritable i= values.next();
            count+= i.get();
        }
        output.collect(key, new IntWritable(count));

    }

}
```

6. Write a Program to Perform Temperature Analysis using Map Reduce

Main

```
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.conf.Configured;
import org.apache.hadoop.io.FloatWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.FileInputFormat;
import org.apache.hadoop.mapred.FileOutputFormat;
import org.apache.hadoop.mapred.JobClient;
import org.apache.hadoop.mapred.JobConf;
import org.apache.hadoop.util.Tool;
import org.apache.hadoop.util.ToolRunner;

public class maxcloseprice extends Configured implements Tool {

    public int run(String[] args) throws Exception {
        if(args.length<2)
        {
            System.out.println("Plz Give Input Output Directory
Correctly");
            return -1;
        }
        JobConf conf = new JobConf(maxcloseprice.class);
        FileInputFormat.setInputPaths(conf,new Path(args[0]));
        FileOutputFormat.setOutputPath(conf, new Path(args[1]));
        conf.setMapperClass(maxclosepricemapper.class);
        conf.setReducerClass(maxclosepricereducer.class);
        conf.setMapOutputKeyClass(Text.class);
        conf.setMapOutputValueClass(FloatWritable.class);
        conf.setOutputKeyClass(Text.class);
        conf.setOutputValueClass(FloatWritable.class);
        JobClient.runJob(conf);
        return 0;
    }
}
```

```
        public static void main(String[] args) throws Exception {
            int exitcode = ToolRunner.run(new maxcloseprice(), args);
            System.exit(exitcode);
        }
    }
```

Mapper

```
import java.io.IOException;
import org.apache.hadoop.io.FloatWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.MapReduceBase;
import org.apache.hadoop.mapred.OutputCollector;
import org.apache.hadoop.mapred.Reporter;
import org.apache.hadoop.mapred.Mapper;

public class maxclosepricemapper extends MapReduceBase implements
Mapper<LongWritable, Text, Text, FloatWritable>
{

    @Override
    public void map(LongWritable key, Text value,
                    OutputCollector<Text, FloatWritable> output, Reporter r)
                    throws IOException {

        String line = value.toString();
        String[] items = line.split(",");

        String stock = items[3];
        if(!items[1].isEmpty() && !items[1].equals(null))
        {
            Float closePrice = Float.parseFloat(items[1]);
            output.collect(new Text(stock), new FloatWritable(closePrice));
        }
    }
}
```

```
}  
}
```

Reducer

```
import java.io.IOException;  
import java.util.Iterator;  
import org.apache.hadoop.io.FloatWritable;  
import org.apache.hadoop.io.Text;  
import org.apache.hadoop.mapred.MapReduceBase;  
import org.apache.hadoop.mapred.OutputCollector;  
import org.apache.hadoop.mapred.Reducer;  
import org.apache.hadoop.mapred.Reporter;  
  
public class maxclosepricereducer extends MapReduceBase implements  
Reducer<Text,FloatWritable,Text,FloatWritable>  
{  
  
    @Override  
    public void reduce(Text key, Iterator<FloatWritable> values,  
        OutputCollector<Text, FloatWritable> output, Reporter r)  
        throws IOException {  
  
        float maxClosePrice=Float.MAX_VALUE;  
        //Iterate all and calculate minimum  
        while (values.hasNext()) {  
            FloatWritable i = values.next();  
            maxClosePrice = Math.min(maxClosePrice, i.get());  
        }  
  
        //Write output  
        output.collect(key, new FloatWritable(maxClosePrice));  
    }  
}
```

7. Analyze Data using Basic Hive Commands

Hive 1---DDL, Create, datatypes (SQL), string, int, varchar, date

Hive 2---Partitions, Buckets

open terminal and type hive

>hive

1. To display the existing databases:

show databases;

2. To create a new database: (Database name is "demo")

create database demo; create database student_163;

3. To select a required database: (Selected database is "demo")

use demo;

4. To create a table in the selected database: (Table name is "student")

create table student(name string, rollno string, age int) row format delimited fields terminated by ',';

5. To insert data into table "student" from a file "Student1" that is stored in local file system:

load data local inpath '/home/training/Desktop/Student1' into table student;

select * from student;

converted to mapreduce jobs

6. To insert some more data into table "student" from a file "Student2" that is stored in local file system:

load data local inpath '/home/training/Desktop/Student2' into table student;

7. To insert some more data into table "student" from a file "Student3" that is stored in local file system: (Student 3 has some field types different from what the schema is defined)

load data local inpath '/home/training/Desktop/Student3' into table student;

8. To display metadata about the table:
describe student;

9. To display detailed metadata about the table:
describe extended student;

10. To insert data into table "student" from a file "Student4" that is stored in hdfs:
First load the data from local file system into hdfs. This can be done using the following command:

```
$ cd desktop
```

```
$ hadoop dfs -put stu4 /student4.txt
```

Then load the data from hdfs into hive table:

```
load data inpath '/student4.txt' into table student;
```

Note: once loaded the file "Student4" will be moved from its path into given hive table "student"

11. To display the contents of table "student":
select count(*) from student;

12. To display the name "Akash" in uppercase:
select upper(name)
from student
where name='Akash';

Execute some more select with aggregate functions

13. To create external table "studextern" in the path "/hivedata":
create external table studextern(
name string,
rollno string,
age int

```
)row format delimited  
fields terminated by ','  
location '/hivedata';
```

14. To insert data into external table "studextern":

```
hadoop dfs -put Student1 /hivedata
```

Note: Just load the file into "/hivedata". All the contents of the file will be inserted into the table "studextern"

15. To display the contents of table "studextern"

```
select * from studextern;
```

Note: Internal table will be in "/user/hive/warehouse"

Note: External table will be in the path where the data is stored.

16. To remove internal table "student":

```
drop table student;
```

17. To remove external table "studextern"

```
drop table studextern;
```

18. To remove database

```
drop database demo;
```

Note: When external table is dropped its schema is dropped but data still exists. Can be checked in browser.

Note: When internal table is dropped its schema and also data is deleted.

8. Analyze data using Advanced HIVE commands Partitioning and Bucketing

1. create database part;

2. use part;

Static Partitioning

3. create table student (name string, rollno int, percentage float)
partitioned by (state string, city string)
row format delimited
fields terminated by ',';

4. load data local inpath '/home/training/Desktop/Maharastra' into table student
partition(state='maharastra', city='mumbai');

5. select * from student;

6. load data local inpath '/home/training/Desktop/karnataka' into table student
partition(state='karnataka', city='Bengaluru');

7. select * from student;

8. select * from student limit 6;

9. select * from student where state='maharastra';

Dynamic partitioning

Note: By default dynamic partitioning will be disabled. We need to enable it using the following command:

10. set hive.exec.dynamic.partition=true;

11. set hive.exec.dynamic.partition.mode=nonstrict;

12. create table stu(name string, rollno int, percentage float, state string, city string)
row format delimited fields terminated by ',';

13. load data local inpath '/home/training/Desktop/Result1' into table stu;

```
14. create table stud_part (name string, rollno int, percentage float)
    partitioned by (state string, city string)
    row format delimited
    fields terminated by ',';
```

```
15. insert overwrite table stud_part
    partition (state, city)
    select name,rollno, percentage
    ,state,
    city
    from stu;
```

```
16. select * from stud_part where city='tumkur';
```

Bucketing

note: To enable bucketing first execute the following command in hive prompt:
hive> set hive.enforce.bucketing=true;

```
1. create database buck;
```

```
2. use buck;
```

```
3. create table employee(id int, fname string, lname string) row format delimited
    fields terminated by ',';
```

```
4. load data local inpath '/home/training/Desktop/emp' into table employee;
```

```
5. create table buck_table(id int, fname string, lname string) clustered by (id) into 5
    buckets row format delimited fields terminated by ',';
```

```
6. insert overwrite table buck_table select * from employee;
```

9. Hbase Commands

Objective: To create and execute hbase commands for the following operations

>hbase shell -- open hbase terminal

1. >status -- status 'simple'
status 'detailed'

>version -- current version

2) Create table

create <tablename>, <columnfamilyname1> <columnfamilyname2>

>create 'emp', 'personal data', 'professional data'

```
put 'emp','1','personal data:name','raju'
put 'emp','1','personal data:city','hyderabad'
put 'emp','1','professional data:designation','manager'
put 'emp','1','professional data:salary','50000'
put 'emp','1','professional data:vechiv','50000'
put 'emp','2','personal data:name','sathish'
put 'emp','2','personal data:city','bangalore'
put 'emp','2','professional data:designation','professor'
put 'emp','2','professional data:salary','60000'
put 'emp','3','personal data:name','muthu'
put 'emp','3','personal data:city','chennai'
put 'emp','3','professional data:designation','analyst'
put 'emp','3','professional data:salary','20000'
```

3. To List all the tables

list

4. To describe the table.

describe 'emp'

5. To delete a table or change its settings, you need to first disable the table using

the disable command.

```
disable 'emp'
```

is_disabled command is used to find whether a table is disabled.

6. To Drop the table

```
drop 'emp'
```

7. To find whether a table is enabled.

```
enable 'emp'
```

The following commands verifies whether the table named **emp** is enabled. If it is enabled, it will return true and if not, it will return false

8. **Scan** command is used to view the data in HBASETable. Using the scan command, you can get the table data.

```
scan 'emp'
```

9 Using the drop command, you can delete a table. Before dropping a table, you have to disable it.

```
hbase(main):018:0> disable 'emp'  
0 row(s) in 1.4580 seconds
```

```
hbase(main):019:0> drop 'emp'  
0 row(s) in 0.3060 seconds
```

10. To get the particular record

```
get method > get 'emp', '1'
```

11. To delete the particular record

```
delete '<table name>', '<row>', '<column name >', '<time stamp>'
```

```
delete 'emp', '1', 'personal data: city',1417521848375
```

12. You can count the number of rows of a table using the count command. Its syntax is as follows:

```
count '<table name>'
count 'emp'
```

13. This command disables drops and recreates a table. The syntax of **truncate** is as follows:

```
hbase> truncate 'table name'

truncate 'emp'
```

Outcome:

Given HBASE commands are created and executed successfully

11. Installation of Spark

1. Boot new/fresh vm image of Ubuntu 14.04,
2. Download jdk1.8 and spark then copy paste jdk and spark package to ubuntu desktop then extract it

Jdk 1.8 link---

https://presidencyuniversityin-my.sharepoint.com/:u:/g/personal/pavithran_presidencyuniversity_in/ERZqi3ymRXFPqLXjslpc_vIBbwWOOGI499RwZr48cIGbMg?e=7LIC9z

Spark link--

https://presidencyuniversityin-my.sharepoint.com/:u:/g/personal/pavithran_presidencyuniversity_in/EfBiiKJUQNRMp5dE0XzKWNyBES54lH-HP6am634vSVX-4A?e=5fPhoA

3. Open the terminal then execute the command:

sudo gedit ~/.bashrc

If asked enter the password as: “password”. Then scroll to the end of the bashrc file and add the following lines into the bashrc file: (This is to set the path of the extracted spark file).

export JAVA_HOME= /home/user/Desktop/jdk1.8.0_333

export PATH=\$PATH:\$JAVA_HOME/bin

export SPARK_PATH=/home/user/Desktop/spark-3.0.1-bin-hadoop2.7

export PATH=\$PATH:\$SPARK_PATH/bin

export PATH=\$PATH:\$SPARK_PATH/sbin

Save the file and close it.

4. Execute the following command: (To reflect the changes done to bashrc file)

source ~/.bashrc

5.Go to desktop , spark-3.0.1-bin-hadoop2.7/bin/load-spark-env.sh

Add this line in line20

export JAVA_HOME= /home/user/Desktop/jdk1.8.0_333

Save and close the file

6. Check java –version in terminal

7. Execute the following command:

ssh-keygen -t rsa -P ""

8. Execute the following command:


```
cat /home/user/.ssh/id_rsa.pub >> /home/user/.ssh/authorized_keys1
```

9. Execute the following command:

```
start-all.sh
```

10. Execute the following command: (It should display “Master” and “Worker” process)

```
Jps
```

11. Then to start Spark shell open the terminal and execute the command:

```
spark-shell
```

12. Word count Spark

1. Open the terminal and execute the command:

```
cd $SPARK_PATH
```

```
sbin/start-all.sh
```

```
jps
```

```
spark-shell
```

```
scala> val text = sc.textFile("file:///home/user/Desktop/test")
```

```
scala> text.collect;
```

```
scala> val counts = text.flatMap(line => line.split(" "))
```

```
scala> counts.collect;
```

```
scala> val mapf = counts.map(word => (word,1))
```

```
scala> mapf.collect;
```

```
scala> val reducef=mapf.reduceByKey(_+_);
```

```
scala> reducef.collect;
```

```
val NUM_SAMPLES=1000
val count = sc.parallelize(1 to NUM_SAMPLES).filter { _ =>
  val x = math.random
  val y = math.random
  x*x + y*y < 1
}.count()
println(s"Pi is roughly ${4.0 * count / NUM_SAMPLES}")
```

```
val textFile1 = sc.textFile("file:///home/user/Desktop/word")
val counts1 = textFile1.flatMap(line => line.split(" "))
val mapf1=counts1.map(word => (word, 1))
val redf1=mapf1.reduceByKey(_ + _)
redf1.saveAsTextFile("file:///home/user/Desktop/word1")
```